

Graduate Algorithms

Quiz 3: Solutions

August 25, 2026

Question 1: Powers of Two, Written Out

6 marks

Suggested/Proposed by *Vardhan Kumar Ray*

You are given the following procedure as a **black box**: you may call it, but you do not need to know how it works.

MULTIPLY(x, y)

- *Input.* The decimal representations of two non-negative integers x and y , having m and n digits respectively, where $m \leq n$.
- *Output.* The decimal representation of the product $x \cdot y$.
- *Running time.* $O(n m^{\lg 3 - 1})$. In particular, when both arguments have d digits this is $O(d^{\lg 3})$.

Assume in addition that adding two d -digit decimal numbers takes $O(d)$ time. The procedure (Algorithm 1) computes the decimal representation of 2^n .

- (a) Write down a recurrence for the running time $T(n)$ of POWER(n), justifying the cost of the work done outside the recursive call. (3 marks)
- (b) Solve your recurrence using a recursion tree. State the bound you get and say which part of the tree carries the dominant cost. (3 marks)

Algorithm 1 Decimal representation of 2^n

```
1: function POWER( $n$ )
2:   if  $n = 0$  then
3:     return 1
4:   end if
5:   if  $n = 1$  then
6:     return 2
7:   end if
8:    $h \leftarrow \lfloor n/2 \rfloor$ 
9:    $x \leftarrow \text{POWER}(h)$                                 ▷ decimal representation of  $2^h$ 
10:   $y \leftarrow \text{MULTIPLY}(x, x)$                           ▷ decimal representation of  $2^{2h}$ 
11:  if  $n$  is odd then
12:     $y \leftarrow y + y$                                     ▷ decimal addition
13:  end if
14:  return  $y$ 
15: end function
```

Solution.

(a) The recurrence

Size of the numbers: A positive integer N has $\lfloor \log_{10} N \rfloor + 1$ decimal digits, so 2^h has $\lfloor h \log_{10} 2 \rfloor + 1 = \Theta(h)$ digits. With $h = \lfloor n/2 \rfloor$ this is $\Theta(n)$ digits.

Work outside the recursive call: There is one call to MULTIPLY on two numbers of $\Theta(n)$ digits each, costing $O(n^{\lg 3})$; and, when n is odd, one addition of numbers of $\Theta(n)$ digits, costing $O(n)$. Since $\lg 3 > 1$, the addition is dominated, and the non-recursive work is $O(n^{\lg 3})$.

Recurrence. There is exactly one recursive call, on $\lfloor n/2 \rfloor$:

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + O(n^{\lg 3}), \quad T(0) = T(1) = O(1).$$

(b) Solving by recursion tree

Since each call makes a single recursive call, the tree is a path: level t holds one node, of size $n/2^t$, at cost $c(n/2^t)^{\lg 3}$ for a constant c . Using $2^{\lg 3} = 3$,

$$\left(\frac{1}{2}\right)^{\lg 3} = \frac{1}{3}, \quad \text{so the cost of level } t \text{ is } cn^{\lg 3} \cdot \frac{1}{3^t}.$$

level t	size	cost of the level
0	n	$cn^{\lg 3}$
1	$n/2$	$cn^{\lg 3}/3$
2	$n/4$	$cn^{\lg 3}/9$
t	$n/2^t$	$cn^{\lg 3}/3^t$

The tree has depth $\lfloor \lg n \rfloor$, and the level costs form a geometric series of ratio $1/3 < 1$, so the sum is within a constant factor of its first term:

$$T(n) \leq cn^{\lg 3} \sum_{t \geq 0} \frac{1}{3^t} = \frac{3}{2} cn^{\lg 3} = O(n^{\lg 3}).$$

The **root** carries the dominant cost: the single top-level multiplication accounts for a constant fraction of the whole running time.

Marking. (a) 1 for 2^h having $\Theta(n)$ digits; 1 for costing the MULTIPLY call at $O(n^{\lg 3})$ and noting the addition is dominated; 1 for the recurrence.

(b) 1 for the level costs; 1 for concluding root dominates; 1 for stating $O(n^{\lg 3})$.

Question 2: The Second Smallest Element

15 marks

Suggested/Proposed by Arjun Maneesh Agarwal, Vardhan Kumar Ray

Let $A[1 \dots n]$ be an array of $n \geq 2$ *distinct* integers. Its *second smallest* element is the element of rank 2, that is, the smallest entry of A apart from the minimum.

We work in the comparison model: an algorithm may compare two entries of A and learn which is smaller, at a cost of one unit, and this is the only way it may inspect the entries. All algorithms below are deterministic.

- Write pseudocode for an algorithm that returns the second smallest element of A in a single pass over the array. State the exact number of comparisons it makes in the worst case, and give its running time. Briefly justify the correctness. **(3+1+1+1 = 6 marks)**
- The algorithm in (a) may not be optimal. Give pseudocode for an algorithm that returns the second smallest element using at most

$$n + \lceil \lg n \rceil - 2$$

comparisons, and show that it makes at most this many comparisons. Briefly justify the correctness. **(5+2+2 = 9 marks)**

Solution.

(a) A single pass

Scan the array once, keeping the smallest and second smallest entries seen so far.

Algorithm 2 Second smallest, one pass

```
1: function SECONDSMALLEST( $A[1 \dots n]$ )
2:   if  $A[1] < A[2]$  then
3:      $min \leftarrow A[1]$ ,  $sec \leftarrow A[2]$ 
4:   else
5:      $min \leftarrow A[2]$ ,  $sec \leftarrow A[1]$ 
6:   end if
7:   for  $i \leftarrow 3$  to  $n$  do
8:     if  $A[i] < min$  then
9:        $sec \leftarrow min$ ,  $min \leftarrow A[i]$ 
10:    else if  $A[i] < sec$  then
11:       $sec \leftarrow A[i]$ 
12:    end if
13:  end for
14:  return  $sec$ 
15: end function
```

Correctness: The invariant is that after the iteration for i , the variables min and sec hold the smallest and second smallest entries of $A[1 \dots i]$. It holds after the initialisation, and the three cases of the loop body cover the three positions $A[i]$ can take relative to min and sec : below both, between them, or above both.

Comparisons: The initialisation costs 1. Each iteration costs 1 if $A[i] < min$ and 2 otherwise, so the worst case is

$$1 + 2(n - 2) = 2n - 3,$$

attained for instance when A is already in increasing order, since then the first test fails every time. The running time is $\Theta(n)$.

(b) A knockout tournament

Run a knockout tournament in which the smaller of two entries wins, recording for each entry the list of entries it has beaten. Then search that list for the winner.

Correctness: Every entry other than m is eliminated exactly once, by the entry that beat it. Let s be the second smallest and suppose z beat s ; then $z < s$, and the only entry smaller than s is m , so $z = m$ and $s \in beaten[m]$. Since $beaten[m] \subseteq A \setminus \{m\}$ and s is the smallest entry of $A \setminus \{m\}$, the smallest entry of $beaten[m]$ is exactly s .

Comparisons. Each comparison eliminates exactly one entry and byes cost nothing, so reducing n survivors to 1 takes exactly $n - 1$ comparisons. A round takes $|S|$ survivors to $\lceil |S|/2 \rceil$, so after r rounds $\lceil n/2^r \rceil$ survive, and the tournament ends after $r = \lceil \lg n \rceil$ rounds. The minimum wins at most one comparison per round, so $|beaten[m]| \leq \lceil \lg n \rceil$, and finding the smallest entry of that list costs $|beaten[m]| - 1 \leq \lceil \lg n \rceil - 1$ comparisons.

The total is therefore at most

$$(n - 1) + (\lceil \lg n \rceil - 1) = n + \lceil \lg n \rceil - 2.$$

Algorithm 3 Second smallest by tournament

```
1: function TOURNAMENTSECOND( $A[1 \dots n]$ )
2:    $S \leftarrow \langle A[1], \dots, A[n] \rangle$  ▷ survivors
3:    $beaten[x] \leftarrow \emptyset$  for every entry  $x$ 
4:   while  $|S| > 1$  do
5:      $S' \leftarrow \langle \rangle$ 
6:     for  $t \leftarrow 1$  to  $\lfloor |S|/2 \rfloor$  do
7:       compare  $S[2t - 1]$  and  $S[2t]$ ; let  $w$  be the smaller and  $\ell$  the larger
8:        $beaten[w] \leftarrow beaten[w] \cup \{\ell\}$ 
9:       append  $w$  to  $S'$ 
10:    end for
11:    if  $|S|$  is odd then
12:      append  $S[|S|]$  to  $S'$  ▷ a bye; no comparison
13:    end if
14:     $S \leftarrow S'$ 
15:  end while
16:   $m \leftarrow S[1]$  ▷ the minimum
17:  return the smallest entry of  $beaten[m]$ 
18: end function
```
